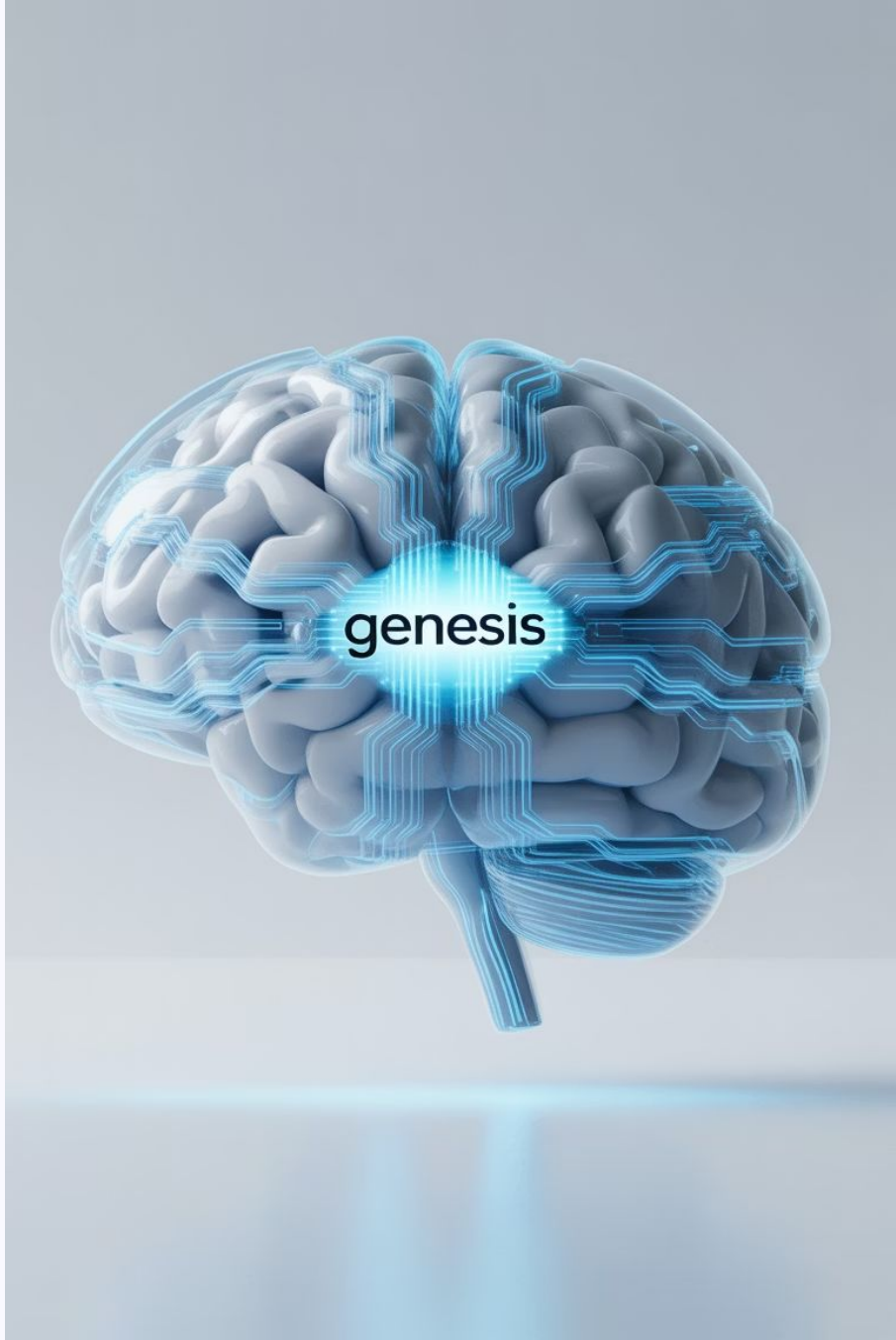
Introduction to Artificial Intelligence CST1440 (Week 5)

Topic: Probability Theory for AI

Taught by: Dr. Ikram Rehman (Dubai)

Associate Professor in AI and Data Science



1 Learning Outcomes

By the end of this week you should be able to:

- Describe sample spaces, events, and basic set operations used in probability theory.
- Calculate probabilities in finite probability spaces, including probabilities of unions, intersections and complements of events.
- Use NumPy arrays and matrix multiplication to perform simple numerical and AI-related computations.
- Generate and analyse random samples using NumPy, and relate simulation results to theoretical probabilities.
- Construct and interpret conditional probabilities and apply the Law of Total Probability.
- Apply Bayes' Theorem to update probabilities in light of new evidence.
- Explain the concept of independence and distinguish it from mutual exclusivity.
- Use conditional independence assumptions to construct and interpret a simple Naive Bayes classifier.

Background, interpretation and motivation

Probability helps us model situations where the outcome is uncertain.
It gives a mathematical way to describe the idea of chance without having to try things many times.

Example

The probability of getting a five on a fair six-sided dice is $\frac{1}{6}$.



There are two common ways to interpret this statement:

1 Frequentist interpretation

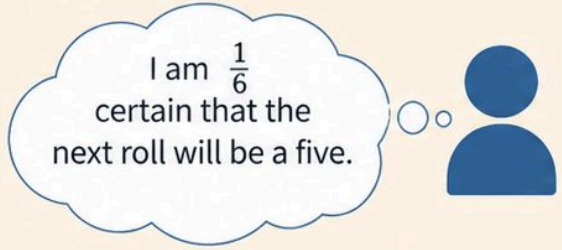
Probability describes what happens in the long run when we repeat the same experiment many times.



If we roll a fair dice many times (e.g. 6000 times), we expect about 1 out of 6 rolls (≈ 1000) to be a five.

2 Bayesian interpretation

Probability describes how confident we are about an outcome.



Both interpretations use the same mathematical framework. They only differ in what the probability means.

3.1 Outcomes and Sample Spaces

We introduce two basic ideas in probability: outcomes and sample spaces.

Example: rolling a six-sided die

If we roll a fair die, we do not know the result in advance. But we do know the result must be one of these six outcomes:

$$\Omega = \{1, 2, 3, 4, 5, 6\}$$



The sample space is the set of all possible outcomes.

Important

- A sample space tells us what can happen.
- It does not tell us the probability yet.

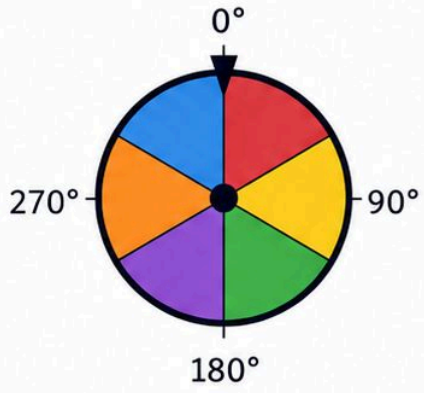
Examples of events

Description	Event
Even number	$\{2, 4, 6\}$
Odd number	$\{1, 3, 5\}$
Less than 4	$\{1, 2, 3\}$
Getting a 3	$\{3\}$

An **event** is a set of outcomes. For example, “even number” means 2 or 4 or 6 on one roll.

Example 2: Spinning a wheel

- If we spin a wheel and measure where it stops as an angle, the result could be 10°, 42.7°, 180°, 359.9°, and many other values.
- So the outcomes are not just whole numbers — there are infinitely many possible results.



The sample space is:

$$\Omega = [0, 360) = \{x \in \mathbb{R} \mid 0 \leq x < 360\}$$

- Ω = all possible outcomes
- $[0, 360)$ = any angle from 0 up to, but not including, 360
- $x \in \mathbb{R}$ = x can be any real number
- $0 \leq x < 360$ = x is between 0 and 360

Why infinitely many outcomes?

- Because the wheel could stop at 12°, 12.5°, 12.53°, 12.531°, …
- There is no end to the possible decimal values.

3.2 Set-theory foundations

- In probability, an **event** is a **set** of outcomes (e.g. throwing an even number).
- A set is **unordered**.
  So $\{2, 4, 6\}$ is the same as $\{6, 2, 4\}$.

$$\{2, 4, 6\} = \{6, 2, 4\}$$

- A set does not "count" how many times an element appears. So $\{2, 4, 4, 4, 6, 4\}$ is the same as $\{2, 4, 6\}$.

$$\{2, 4, 4, 4, 6, 4\} = \{2, 4, 6\}$$

Tip

Python distinguishes between lists and sets.

A `list` is ordered:

```
[2,4,6] == [6,2,4]
```

```
False
```

and multiple entries are respected:

```
[2,4,6] == [2,4,4,6]
```

```
False
```

A `set` is unordered:

```
set([2,4,6]) == set([6,2,4])
```

```
True
```

and multiple entries are ignored:

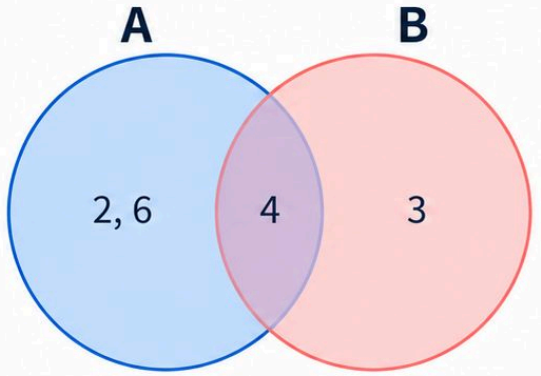
```
set([2,4,6]) == set([2,4,4,4,6,4])
```

```
True
```

Event operations: Intersection (and)

- We can combine events using the **and** operation.
- The and operation, written as $\cap$, keeps only the outcomes that are in **both** events.
- For example,
  - $A = \{2, 4, 6\}$ (even numbers)
  - $B = \{3, 4\}$ (numbers less than 5)
- The intersection ($A \cap B$) is

$$\{2, 4, 6\} \cap \{3, 4\} = \{4\}$$



In Python:

```
A = set([2,4,6])
B = set([3,4])
print(A.intersection(B))
```

```
{4}
```

A ∩ B means "A **and** B": outcomes that are in **both** events.

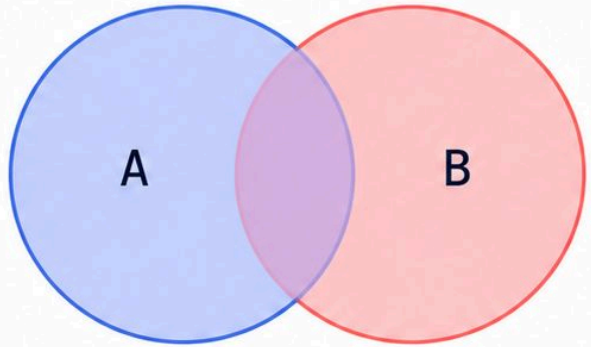
4 Event operations

Similarly, the **or** ($\vee$) operation:

$$\{2, 4, 6\} \vee \{3, 4\} = \{2, 3, 4, 6\}$$

The **or** operation keeps all outcomes that are in A, in B, or in both.

We only write 4 once because sets do not repeat elements.



$A \vee B$

In Python:

```
A = set([2,4,6])
B = set([3,4])
print(f"The union of A and B is {A.union(B)}")
```

```
The union of A and B is {2, 3, 4, 6}
```

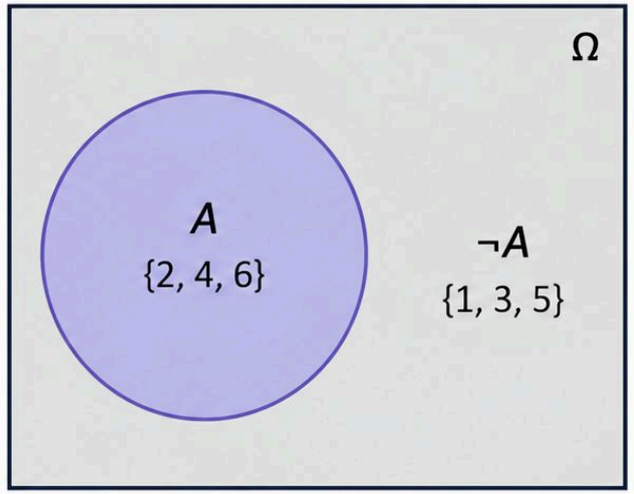
Complement of an event (not)

- The **complement** of an event A, written as A^c or $\neg A$, contains all outcomes in the sample space that are **not** in A.
- If the sample space is

$$\Omega = \{1, 2, 3, 4, 5, 6\}$$

- and $A = \{2, 4, 6\}$ (even numbers), then

$$\neg A = \{1, 3, 5\}$$



The complement includes all outcomes in the sample space that are **not** in the event.

Implication with Events

1 Core idea

'If A then B' means **every** outcome in A is **also** in B.
In set notation, we show this as: $A \subset B$

2 Dice example

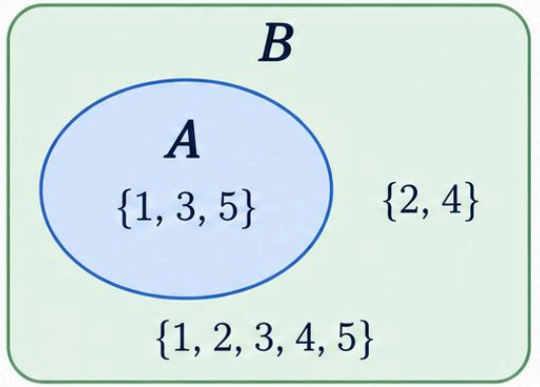
Example: rolling a six-sided die.

A = odd number
$A = \{1, 3, 5\}$

B = less than 6
$B = \{1, 2, 3, 4, 5\}$

$$A \subset B$$

Every value in A is inside B, so 'if A then B' is true.



3 Important note

! $A \subset B$ is **not** true if A contains even one outcome that is not in B.

4.1 Uniform probability: “fairness”

What does “fair” mean?

- A situation is fair when all outcomes are **equally likely to occur**.
- There is no reason for one outcome to be more likely than another.
- Example: a fair six-sided dice, where each face 1, 2, 3, 4, 5, 6 has the same chance.

Example: fair six-sided dice



Sample space:
$\Omega = \{1, 2, 3, 4, 5, 6\}$

Each outcome has probability 1/6.

Uniform (fair) probability

- For a **finite** sample space with **equally likely outcomes**, the probability of an **event** A is:

$$P(A) = \frac{\text{number of outcomes in } A}{\text{number of outcomes in } \Omega}$$

Example

Let A be the event “throw an even number”.
$A = \{2, 4, 6\}$ has 3 outcomes, and Ω has 6 outcomes.

$$P(A) = \frac{3}{6} = \frac{1}{2}$$

4.2 Basic probability rules

Not all situations are fair. Sometimes outcomes do not have equal probabilities.

Example: {win the lottery, not win the lottery}

There are 2 outcomes, but they do not each have probability $\frac{1}{2}$.

Three rules to remember

1. A probability cannot be negative. $P(E) \geq 0$
2. The probability of the whole sample space is 1. $P(\Omega) = 1$
3. If two events cannot happen together (mutually exclusive), add their probabilities. $P(E \text{ or } F) = P(E) + P(F)$

Useful results

1. Probabilities are always between 0 and 1. $0 \leq P(E) \leq 1$
2. Probability of not E. $P(\text{not } E) = 1 - P(E)$
3. If events overlap: $P(E \text{ or } F) = P(E) + P(F) - P(E \text{ and } F)$
   Use this when E and F can happen together.

5 NumPy: quick start

What is NumPy?

- NumPy is a Python library for numerical computations.
- It provides fast and easy-to-use functions for mathematics, arrays and data analysis.
- It is widely used in AI, data science and machine learning.

Importing NumPy

```
import numpy as np
```

- We usually import NumPy with the short name `np`.

Examples

1 Pi (π)

```
import numpy as np
np.pi
```

NumPy provides the constant π.

```
3.141592653589793
```

2 Sine function

```
import numpy as np
np.sin(np.pi / 4)
```

Compute sin(π/4) (using radians).

```
0.7071067811865476
```

5.1 Arrays

NumPy works very well with collections of numbers called **arrays.**

1 Start with a Python list

```
list_vals = [1, 5, 7, 4]
print(list_vals)
```

Output:

```
[1, 5, 7, 4]
```

2 Turn it into a NumPy array

```
array_vals = np.array([1, 5, 7, 4])
print(array_vals)
```

Output:

```
[1 5 7 4]
```

3 They look similar, but they behave differently

Python list + list

```
print(list_vals + list_vals)
```

Output:

```
[1, 5, 7, 4, 1, 5, 7, 4]
```

Lists join together.

NumPy array + array

```
print(array_vals + array_vals)
```

Output:

```
[ 2 10 14  8]
```

Arrays add values position by position.



Key takeaway: Array = numbers stored for fast numerical work. This is why NumPy is useful in AI and data science.

5.1 NumPy Arrays (continued)

1 Arrays can do maths easily

NumPy applies the operation to every element in the array.

Python code

```
print(array + 5)      # add 5 to every element
print(array * 4)      # multiply every element by 4
print(np.cos(array)) # cosine of every element
```

Output

```
[ 6 10 12  9 ]
[ 4 20 28 16 ]
[ 0.54030231  0.28366219  0.75390225 -0.65364362 ]
```

2 Generating arrays with np.arange()

np.arange(start, stop, step) creates a sequence of numbers.

Python code

```
x = np.arange(start = 4, stop = 20, step = 0.2)
print(x)
```

Output (simplified)

```
[4.0, 4.2, 4.4, 4.6, ..., 19.4, 19.6, 19.8]
```

Important: the stop value is not included.

Unlike range(), np.arange() can use decimal steps.

3 Arrays can have more than one dimension

A 2D array is like a table with rows and columns.

Python code

```
A = np.array([[1, 3, 7], [2, 4, 6]])
print(A)
```

Output

```
[[1 3 7]
 [2 4 6]]
```

Python code

```
A.shape
```

Output

```
(2, 3)
```

This means 2 rows and 3 columns.

Takeaway: NumPy arrays are useful because they let us store numbers efficiently and perform calculations quickly.

5.2 Array multiplication

```
A = np.array([1, 2, 3, 4])
B = np.array([5, 10, 5, 10])
```

1 Scalar multiplication

Multiply every value in *A* by 4.

```
print(4 * A)
```

Result:

```
[ 4  8 12 16 ]
```

2 Elementwise multiplication

Multiply matching positions in the two arrays.

```
print(A * B)
```

Result:

```
[ 5 20 15 40 ]
```

1×5 2×10 3×5 4×10

3 The @ operator for 1D arrays

For 1D arrays, `@` gives the dot product. Multiply matching values, then add them.

```
print(A @ B)
```

Result:

```
80
```

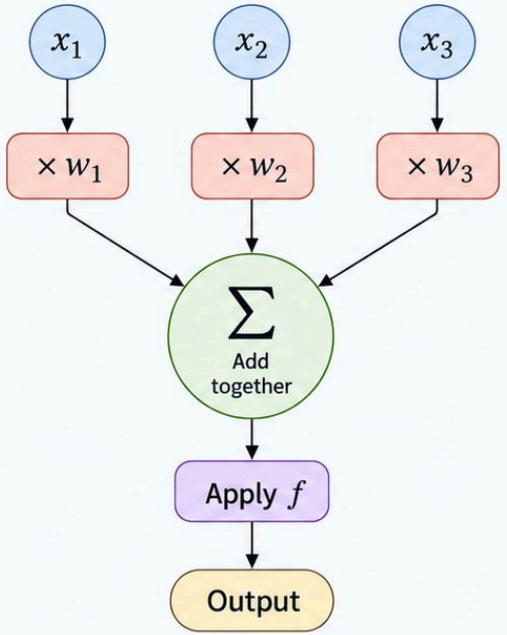
Calculation:

$1 \times 5 + 2 \times 10 + 3 \times 5 + 4 \times 10 = 80$

Takeaway: | `A * B` keeps the separate products. `A @ B` adds them together.

Artificial Neuron with NumPy

1. Artificial neuron



2. Single input

```python
import numpy as np
x = np.array([1, 0, 1])   # input vector
w = np.array([4, 7, 1])   # weight vector

z = x @ w                 # weighted sum
f = (z >= 8).astype(int)  # activation
print(z)   # 5
print(f)   # 0
```

Weighted sum:

$$x \mathbin{@} w = 5$$

Activation function:

$$f(z) = \begin{cases} 0 & z < 8 \\ 1 & z \geq 8 \end{cases}$$

Result: $f(x \mathbin{@} w) = 0$

The neuron does not fire.

3. Multiple inputs at once

```python
import numpy as np
x = np.array([[1, 0, 1],
              [1, 1, 0]])  # two inputs (rows)
w = np.array([4, 7, 1])    # weight vector

z = x @ w                  # weighted sums: [5, 8]
f = (z >= 8).astype(int)   # apply activation
print(f)   # [0, 1]
```

Result:

$$f(x \mathbin{@} w) = [0,\ 1]$$

First input: no fire.
Second input: fires.

Key takeaway: Array multiplication helps us compute neuron outputs efficiently, especially when we have many inputs.

6 Random Number Generation

NumPy can generate random numbers for simulation and probability experiments.

1 Older code you may still see

```
np.random.randint(1, 7)
```

Gives a random whole number from 1 to 6.

2 Preferred modern approach

```
rng = np.random.default_rng()
```

Create a random number generator.

```
rng.integers(1, 7, size=10)
```

Generate 10 random whole numbers from 1 to 6.

```
rng.random()
```

Generate one random decimal between 0 and 1.

In short: older code often uses `np.random.randint()`, but for new code we usually use `rng = np.random.default_rng()` and then call methods such as `rng.integers()` or `rng.random()`.

7 Finite probability spaces

1 What is a finite probability space?

- A finite probability space has a **limited number** of possible outcomes.
- Each outcome is given a probability (also called a **weight**).
- The probabilities should **add up to 1.**

2 Example: degree outcomes

Suppose we want to model the outcome of a student's degree.

Sample space (all possible outcomes):
{First class, Second class, Third class, Not graduate}

Based on historical data, we assign the following probabilities:

Outcome	Probability
First class	0.30
Second class	0.4995
Third class	0.20
Not graduate	0.0005

3 Question

What is the probability that a student graduates?

$$P(\text{graduate}) = 0.30 + 0.4995 + 0.20$$
$$= \mathbf{0.9995}$$

4 Python: randomly choose a degree outcome

```
outcomes = ['First class', 'Second class', 'Third class', 'Not graduate']   # possible outcomes
weights = [0.3, 0.4995, 0.2, 0.0005]   # probability of each outcome

rng.choice(outcomes, p = weights)    # randomly pick one outcome using these probabilities
```

- This code randomly chooses one outcome from the list. The values in `weights` tell Python how likely each outcome is.
- Example output: `'Second class'`

8 Conditional probability and probability table

1 What is conditional probability?

- The conditional probability of an event F occurring given that another event E has occurred.
- It allows us to update our beliefs when we have new information.

$$P(F \mid E) = \frac{P(F \cap E)}{P(E)}$$

We write $P(F \mid E)$ and say "the probability of F **given** E".

2 Example: spam detection

Suppose we want to classify emails as spam or not spam, using a simple rule: check if the email contains the word "lottery".

	Spam	Not-spam
Contains "lottery"	0.14	0.04
Doesn't contain "lottery"	0.06	0.76

Each cell gives the probability of the intersection.

- For example, $P(\text{Contains "lottery"} \cap \text{Spam}) = 0.14$.

3 Example 1: probability of spam given the word "lottery"

What is the probability that an email is spam, given that it contains "lottery"?

$$P(\text{Spam} \mid \text{lottery}) = \frac{P(\text{Spam} \cap \text{lottery})}{P(\text{lottery})}$$

$$P(\text{lottery}) = 0.14 + 0.04 = 0.18$$

$$P(\text{Spam} \mid \text{lottery}) = \frac{0.14}{0.18} = 0.778$$

If an email contains "lottery", the probability that it is spam is 0.778 (about 78%).

4 Example 2: probability of the word "lottery" in spam emails

What is the probability that a spam email contains "lottery"?

$$P(\text{lottery} \mid \text{Spam}) = \frac{P(\text{lottery} \cap \text{Spam})}{P(\text{Spam})}$$

$$P(\text{Spam}) = 0.14 + 0.06 = 0.20$$

$$P(\text{lottery} \mid \text{Spam}) = \frac{0.14}{0.20} = 0.70$$

If an email is spam, the probability that it contains "lottery" is 0.70 (70%).

9.2 Bayes' Theorem – Vaccine Example

1 Part 1: Given information (before seeing the trial)

A friend (Alex) strongly believes that vaccines **do not work**.

We write this belief as:

$$P(B) = 0.99$$

B = vaccines do not work

Alex is 99% sure that vaccines do not work.

So the opposite belief is:

$$P(not\ B) = 1 - 0.99 = 0.01$$

not B = vaccines do work

Alex is only **1% sure** that vaccines do work.

2 What does the trial (new data) show?

A vaccine trial gives the following results:

If vaccines **do not work** (B), the chance of seeing this data is very small – only 2%.

$$P(\text{Data} \mid B) = 0.02$$

If vaccines **do work** (*not* B), the chance of seeing this data is high – about 80%.

$$P(\text{Data} \mid not\ B) = 0.80$$

Data = the observed result from the trial (e.g. vaccinated people get sick much less often).

3 Step 1: Find the overall probability of the data

The trial result could occur in two ways: vaccines do not work or they do work.

$$P(\text{Data}) = P(\text{Data} \mid B)P(B) + P(\text{Data} \mid not\ B)P(not\ B)$$

Substitute the values:

$$\begin{aligned} P(\text{Data}) &= (0.02 \times 0.99) + (0.80 \times 0.01) \\ &= 0.0198 + 0.0080 \\ &= \mathbf{0.0278} \end{aligned}$$

So, overall, there is about a **2.78% chance** of seeing this trial result at all.

4 Step 2: Update the belief using Bayes' Theorem

We now want to know: after seeing this trial result, what is the new probability that vaccines do not work?

$$P(B \mid \text{Data}) = \frac{P(\text{Data} \mid B) \times P(B)}{P(\text{Data})}$$

Substitute the values:

$$P(B \mid \text{Data}) = \frac{0.02 \times 0.99}{0.0278} = \frac{0.0198}{0.0278} = \mathbf{0.712}$$

5 Conclusion

After seeing the trial result (data), Alex's belief that vaccines do not work updates **from 99% to 71%**.

The data provides strong evidence against Alex's initial belief, but they may still believe vaccines do not work (71%).

10. Independence of Events

1 What does independence mean?

Two events are independent if the occurrence of one **does not change** the probability of the other.

$$P(E \cap F) = P(E)P(F)$$

or equivalently

$$P(E \mid F) = P(E)$$

Knowing that event F happened gives no extra information about the chance of event E happening.

2 Real example: Weather and a coin toss

Let's consider two events:

- E = It rains today
- F = A fair coin shows heads



Whether it rains or not does not affect the coin toss result, and the coin toss result does not affect the weather.

So, these two events are independent.

3 Check with numbers

Suppose:

- $P(\text{It rains}) = 0.30$ (30%)
- $P(\text{Heads}) = 0.50$ (fair coin)

Since the events are independent:

- $P(\text{It rains and heads}) = 0.30 \times 0.50 = 0.15$
- $P(\text{It rains} \mid \text{heads}) = 0.30$
- $P(\text{Heads} \mid \text{it rains}) = 0.50$

4 Contrast: A dependent example

Consider two events:

- E = It is cloudy
- F = It rains



These events are **not independent.**
If it is cloudy, the chance of rain is much higher:
$P(\text{Rain} \mid \text{Cloudy}) > P(\text{Rain})$

1 What is Naive Bayes?

- **"Bayes"** = use known probabilities to make a prediction.
- **"Naive"** = assume the clues (words) act independently.
- Here, the clues are the words in the email.

In simple terms:
Which class makes this email more likely — Spam or Not Spam?

2 How the spam example works

Example email: "Urgent: your winning lottery ticket is about to expire."

Known from past emails	Spam	Not Spam
How common?	0.20	0.80
Word "lottery" appears	0.70	0.05
Word "urgent" appears	0.60	0.10

1. **Start with** how common each class is.
2. **For Spam,** combine the clue probabilities:
   $0.20 \times 0.70 \times 0.60 = \mathbf{0.084}$
3. **For Not Spam,** combine the clue probabilities:
   $0.80 \times 0.05 \times 0.10 = \mathbf{0.004}$
4. **Compare the two scores.**
   0.084 is much bigger than 0.004, so predict: **Spam**

Why multiply? Because we are combining the chance of seeing all these clues together for each class, using the Naive assumption.

Higher score = more likely class.

Questions

